



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

Факультет

«Информатика и системы управления»

Кафедра

«Системы обработки информации и управления»

Отчет по лабораторной работе №5

«Ансамбли моделей машинного обучения. Часть 1.»

по дисциплине

«Технологии машинного обучения»

Выполнил:
Сорокин М.А. ИУ5-63Б

Проверил:
преподаватель Ю.Е. Гапанюк

2026 г.

Лабораторная работа: Ансамбли моделей машинного обучения

(Часть 1)

Цель: изучить ансамбли моделей машинного обучения на задаче классификации.

Датасет: Bank_Churn.csv .

В работе будут обучены и сравнены 4 ансамблевые модели:

- RandomForestClassifier (группа бэггинга)
- ExtraTreesClassifier (группа бэггинга)
- AdaBoostClassifier
- GradientBoostingClassifier

```
In [1]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

from sklearn.ensemble import (
    RandomForestClassifier,
    ExtraTreesClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier,
)
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

RANDOM_STATE = 42
```

```
In [2]: # 1) Загрузка данных

df = pd.read_csv('Bank_Churn.csv')

print('Размер датасета:', df.shape)
df.head()
```

Out:

Размер датасета: (10000, 13)

	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1	101334.36
1	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	113522.17
2	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0	113522.17
3	15701354	Boni	699	France	Female	39	1	0.00	2	0	0	92674.03
4	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	79043.41

```
In [3]: # 2) Базовый анализ и подготовка

print('Пропуски по столбцам:')
print(df.isna().sum())

# Целевая переменная
```

```
TARGET = 'Exited'

# Служебные/идентификационные признаки исключаем
DROP_COLS = ['CustomerId', 'Surname']

X = df.drop(columns=[TARGET] + DROP_COLS)
y = df[TARGET]

print('\nРаспределение целевого класса:')
print(y.value_counts(normalize=True).rename('share'))

# Категориальные и числовые признаки
cat_features = X.select_dtypes(include=['object', 'string']).columns.tolist()
num_features = [col for col in X.columns if col not in cat_features]

print('\nКатегориальные признаки:', cat_features)
print('Числовые признаки:', num_features)
```

Out:

```
Пропуски по столбцам:
CustomerId      0
Surname         0
CreditScore     0
Geography       0
Gender          0
Age             0
Tenure          0
Balance         0
NumOfProducts  0
HasCrCard       0
IsActiveMember  0
EstimatedSalary 0
Exited          0
dtype: int64

Распределение целевого класса:
Exited
0      0.7963
1      0.2037
Name: share, dtype: float64

Категориальные признаки: ['Geography', 'Gender']
Числовые признаки: ['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary']
```

In [4]: # 3) Разделение на обучающую и тестовую выборки

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y,
)

print('Train shape:', X_train.shape)
print('Test shape:', X_test.shape)
```

Out:

```
Train shape: (8000, 10)
Test shape: (2000, 10)
```

In [5]: # 4) Предобработка
Добавляем обработку пропусков и кодирование категориальных признаков.
В текущем датасете пропусков нет, но шаг оставлен для корректности по заданию.

```
numeric_transformer = Pipeline(
    steps=[('imputer', SimpleImputer(strategy='median'))]
```

```

)

categorical_transformer = Pipeline(
    steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, num_features),
        ('cat', categorical_transformer, cat_features),
    ]
)

```

In [6]: # 5) Обучение ансамблевых моделей

```

models = {
    'RandomForest (Bagging)': RandomForestClassifier(
        n_estimators=300,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
    'ExtraTrees (Bagging)': ExtraTreesClassifier(
        n_estimators=300,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
    'AdaBoost': AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE),
        n_estimators=300,
        learning_rate=0.05,
        random_state=RANDOM_STATE,
    ),
    'GradientBoosting': GradientBoostingClassifier(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=3,
        random_state=RANDOM_STATE,
    ),
}

results = []

for name, model in models.items():
    pipe = Pipeline(
        steps=[
            ('preprocessor', preprocessor),
            ('model', model),
        ]
    )

    pipe.fit(X_train, y_train)

    y_pred = pipe.predict(X_test)
    y_proba = pipe.predict_proba(X_test)[: , 1]

    results.append(
        {
            'Model': name,
            'Accuracy': accuracy_score(y_test, y_pred),
            'F1': f1_score(y_test, y_pred),
            'ROC-AUC': roc_auc_score(y_test, y_proba),
        }
    )

```

```
results_df = pd.DataFrame(results).sort_values('ROC-AUC', ascending=False).reset_index(drop=True)
results_df
```

Out:

	Model	Accuracy	F1	ROC-AUC
0	GradientBoosting	0.870	0.609610	0.870109
1	RandomForest (Bagging)	0.858	0.559006	0.849270
2	ExtraTrees (Bagging)	0.856	0.551402	0.845403
3	AdaBoost	0.819	0.206140	0.836108

In [7]:

```
# 6) Итоговое сравнение и вывод

best_model = results_df.iloc[0]
print('Лучшая модель по ROC-AUC:')
print(best_model)

print('\nКраткий вывод:')
print(
    'По результатам эксперимента сравнение ансамблей показывает, '
    'что бустинговые и бэггинговые подходы дают сопоставимо высокое качество, '
    'но лидер определяется по метрике ROC-AUC на тестовой выборке.'
)
```

Out:

```
Лучшая модель по ROC-AUC:
Model      GradientBoosting
Accuracy           0.87
F1              0.60961
ROC-AUC         0.870109
Name: 0, dtype: object

Краткий вывод:
По результатам эксперимента сравнение ансамблей показывает, что бустинговые и бэггинговые подходы дают сопоставимо высокое качество, но лидер определяется по метрике ROC-AUC на тестовой выборке.
```

Что выполнено по заданию

- Выбран датасет для задачи **классификации**.
- Выполнена подготовка признаков: исключены ID-поля, добавлены шаги обработки пропусков и кодирования категорий.
- Использован `train_test_split` для деления на train/test.
- Обучены 4 ансамблевые модели:
 - две модели группы бэггинга (`RandomForest` , `ExtraTrees`),
 - `AdaBoost` ,
 - `GradientBoosting` .
- Проведена оценка качества (Accuracy, F1, ROC-AUC) и сравнение моделей.